

CITY2CRUISE

Shop&Drop Port Hub

HOJA DE RUTA DE DESARROLLO

De Alpha a Producción: Guía Completa de Hitos y Fases

Versión 1.0 | Abril 2026

Documento Confidencial

Equipo de Desarrollo: Luis & Pablo



ÍNDICE DE CONTENIDOS

ÍNDICE DE CONTENIDOS	2
1. RESUMEN EJECUTIVO	5
1.1 Estado Actual del Proyecto.....	5
1.2 Tabla de Estado Actual	5
1.3 Visión General de Fases	5
2. FASE 1: ESTABILIZACIÓN Y DEUDA TÉCNICA	7
2.1 Hito 1.1: Gestión de Estado del Frontend	7
Tareas:	7
Criterios de aceptación:.....	7
2.2 Hito 1.2: Manejo de Errores del Frontend	7
Tareas:	7
Criterios de aceptación:.....	8
2.3 Hito 1.3: Refactorización de Componentes Principales.....	8
Tareas:	8
Criterios de aceptación:.....	8
2.4 Hito 1.4: Estabilización del Backend	9
Tareas:	9
Criterios de aceptación:.....	9
3. FASE 2: SEGURIDAD Y CUMPLIMIENTO	10
3.1 Hito 2.1: Fortalecimiento de Autenticación	10
Tareas:	10
Criterios de aceptación:.....	10
3.2 Hito 2.2: Protección Anti-Spoofing GPS.....	10
Tareas:	10
Criterios de aceptación:.....	11
3.3 Hito 2.3: Seguridad de Datos y Comunicaciones	11
Tareas:	11
Criterios de aceptación:.....	11
3.4 Hito 2.4: Cumplimiento GDPR/RGPD	11
Tareas:	11
Criterios de aceptación:.....	12
4. FASE 3: INTEGRACIONES EXTERNAS	13
4.1 Hito 3.1: Pasarela de Pagos	13
Tareas:	13
Criterios de aceptación:.....	13
4.2 Hito 3.2: Sistema de Notificaciones SMS/Push.....	13

Tareas:	13
Criterios de aceptación:.....	14
4.3 Hito 3.3: Integración con Lockers IoT	14
Tareas:	14
Criterios de aceptación:.....	14
4.4 Hito 3.4: Integración con Líneas de Cruceros	15
Tareas:	15
Criterios de aceptación:.....	15
5. FASE 4: EXPERIENCIA DE USUARIO Y RENDIMIENTO	16
5.1 Hito 4.1: Accesibilidad (WCAG 2.1 AA).....	16
Tareas:	16
Criterios de aceptación:.....	16
5.2 Hito 4.2: Optimización de Rendimiento Frontend	16
Tareas:	16
Criterios de aceptación:.....	17
5.3 Hito 4.3: Optimización de Rendimiento Backend	17
Tareas:	17
Criterios de aceptación:.....	17
6. FASE 5: INFRAESTRUCTURA Y DEVOPS	18
6.1 Hito 5.1: Pipeline CI/CD	18
Tareas:	18
Criterios de aceptación:.....	18
6.2 Hito 5.2: Infraestructura Cloud	18
Tareas:	18
Criterios de aceptación:.....	19
6.3 Hito 5.3: Monitorización y Observabilidad	19
Tareas:	19
Criterios de aceptación:.....	19
7. FASE 6: TESTING INTEGRAL Y QA	20
7.1 Hito 6.1: Tests Unitarios del Frontend	20
Tareas:	20
Criterios de aceptación:.....	20
7.2 Hito 6.2: Tests End-to-End (E2E)	20
Tareas:	20
Criterios de aceptación:.....	21
7.3 Hito 6.3: Tests de Seguridad	21
Tareas:	21
Criterios de aceptación:.....	21
7.4 Hito 6.4: Tests de Carga y Estrés	21

Tareas:	21
Criterios de aceptación:.....	22
8. FASE 7: LANZAMIENTO Y POST-LANZAMIENTO	23
8.1 Hito 7.1: Preparación Pre-Lanzamiento	23
Tareas:	23
Criterios de aceptación:.....	23
8.2 Hito 7.2: Lanzamiento Beta Controlado	23
Tareas:	23
Criterios de aceptación:.....	24
8.3 Hito 7.3: Lanzamiento Público (GA)	24
Tareas:	24
Criterios de aceptación:.....	24
8.4 Hito 7.4: Plan Post-Lanzamiento.....	24
Tareas:	24
Criterios de aceptación:.....	25
9. CRONOGRAMA CONSOLIDADO	26
9.1 Equipo: Luis y Pablo	26
9.2 Dinámica de Trabajo Luis-Pablo	27
10. GESTIÓN DE RIESGOS.....	28
11. MÉTRICAS DE ÉXITO (KPIs)	29
11.1 KPIs Técnicos	29
11.2 KPIs de Negocio.....	29

1. RESUMEN EJECUTIVO

City2Cruise Shop&Drop Port Hub es una plataforma de logística de última milla diseñada para conectar cruceristas y viajeros de puerto con un servicio de recogida, transporte y depósito de equipaje y compras en consignas inteligentes (lockers). La aplicación se encuentra actualmente en fase alpha con aproximadamente un 65-70% de preparación para producción.

Este documento establece la hoja de ruta completa para llevar la aplicación desde su estado alpha actual hasta un producto completamente funcional, seguro, estable y listo para su lanzamiento comercial. Se estructura en 7 fases progresivas, cada una con hitos concretos, criterios de aceptación y estimaciones de esfuerzo. El proyecto será ejecutado por Luis y Pablo, que trabajarán compenetrados como equipo de desarrollo, cubriendo conjuntamente todas las áreas necesarias: frontend, backend, DevOps, seguridad, testing y lanzamiento.

1.1 Estado Actual del Proyecto

El proyecto cuenta con una arquitectura sólida basada en React 18 + TypeScript en el frontend, Node.js + Express 5 en el backend, y PostgreSQL 15 con extensión PostGIS para funcionalidad geoespacial. El flujo core de transacción (solicitud de recogida, handshake conductor-cliente, depósito en locker y recogida final) está implementado y funcional.

1.2 Tabla de Estado Actual

Componente	Estado	Cobertura / Notas
Flujo transaccional core	Funcional	5 estados del ciclo de vida implementados
Autenticación JWT + bcrypt	Funcional	HS256, expiración 24h, 3 roles
Base de datos PostgreSQL + PostGIS	Funcional	8 tablas, índices, geoqueries
Tests backend	Funcional	13 suites, 71% cobertura de líneas
Tests frontend	Crítico	Menos del 10% de cobertura
Tests E2E	Inexistente	No hay Cypress/Playwright
Pasarela de pagos	No implementado	Sin integración Stripe/Adyen
Envío SMS/OTP	No implementado	Código solo visible en app
Integración IoT lockers	No implementado	API simulada
CI/CD Pipeline	No configurado	Carpeta GitHub Actions vacía
Monitorización	No implementado	Sin Sentry/Datadog
Seguridad GPS anti-spoofing	No implementado	Documentado pero no codificado

1.3 Visión General de Fases

Fase	Nombre	Duración Estimada	Prioridad
1	Estabilización y Deuda Técnica	3-4 semanas	CRÍTICA

Fase	Nombre	Duración Estimada	Prioridad
2	Seguridad y Cumplimiento	2-3 semanas	CRÍTICA
3	Integraciones Externas	4-5 semanas	ALTA
4	Experiencia de Usuario y Rendimiento	3-4 semanas	ALTA
5	Infraestructura y DevOps	2-3 semanas	ALTA
6	Testing Integral y QA	3-4 semanas	CRÍTICA
7	Lanzamiento y Post-Lanzamiento	2-3 semanas	ALTA

2. FASE 1: ESTABILIZACIÓN Y DEUDA TÉCNICA

Duración estimada: 3-4 semanas | **Prioridad:** CRÍTICA

Esta fase se centra en resolver los problemas fundamentales de estabilidad y calidad de código que impiden avanzar con confianza hacia producción. Sin completar esta fase, cualquier funcionalidad nueva se construirá sobre cimientos frágiles.

2.1 Hito 1.1: Gestión de Estado del Frontend

Problema actual: La aplicación utiliza sessionStorage como mecanismo de persistencia de estado, lo que provoca pérdida total de datos al refrescar el navegador. Esto es inaceptable para una aplicación de producción.

Tareas:

1. **Implementar estado global robusto:** Migrar de sessionStorage + Context API a una solución como Zustand o Redux Toolkit con persistencia configurable. Zustand es recomendado por su menor huella y compatibilidad con React 18.
2. **Persistencia selectiva:** Configurar persistencia en localStorage (o IndexedDB para datos mayores) solo para datos críticos: token de autenticación, estado de solicitud activa, preferencias del usuario. Los datos sensibles como PINs de locker nunca deben persistirse localmente.
3. **Rehidratación de sesión:** Implementar lógica que al recargar la página valide el token JWT con el backend, recupere el estado actual de la solicitud activa y reestablezca las conexiones Socket.IO automáticamente.
4. **Manejo de tokens expirados:** Implementar interceptor de Axios que detecte respuestas 401, limpie el estado local, y redirija al login con mensaje informativo.

Criterios de aceptación:

- El usuario puede refrescar la página sin perder su sesión activa ni el estado de su solicitud en curso.
- Al expirar el token JWT, el usuario es redirigido al login automáticamente con un mensaje claro.
- No se almacenan datos sensibles (PINs, códigos OTP) en almacenamiento del navegador.

2.2 Hito 1.2: Manejo de Errores del Frontend

Problema actual: No existen React Error Boundaries, lo que significa que un error de JavaScript en cualquier componente puede provocar una pantalla en blanco irre recuperable. El manejo de errores de red es genérico e insuficiente.

Tareas:

1. **Error Boundaries globales:** Implementar un ErrorBoundary de nivel superior que capture errores de renderizado y muestre una interfaz de recuperación amigable con opción de recargar.
2. **Error Boundaries por sección:** Añadir boundaries específicos para cada dashboard (Cliente, Conductor, Admin) de modo que un error en una sección no colapse las demás.

3. **Manejo de errores de red:** Implementar lógica de reintentos con backoff exponencial en las llamadas API críticas (aceptar solicitud, confirmar handshake, depósito). Configurar timeouts explícitos en Axios (10 segundos para operaciones normales, 30 para carga de datos pesados).
4. **Estado offline:** Detectar pérdida de conexión y mostrar un banner persistente informando al usuario. Encolar operaciones críticas para reintento cuando se recupere la conexión.
5. **Logging de errores:** Integrar un servicio de reporting de errores del lado del cliente (preparar la infraestructura para Sentry que se configurará completamente en Fase 5).

Criterios de aceptación:

- Ningún error de JavaScript produce una pantalla en blanco. Siempre se muestra una UI de recuperación.
- Las operaciones críticas reintentan automáticamente hasta 3 veces con backoff exponencial.
- El usuario ve un indicador claro cuando está sin conexión.

2.3 Hito 1.3: Refactorización de Componentes Principales

Problema actual: Los dashboards principales son archivos monolíticos excesivamente grandes (ClientDashboard.tsx tiene 29KB, AdminDashboard.tsx tiene 17KB). Esto dificulta el mantenimiento, el testing y la revisión de código.

Tareas:

1. **Descomponer ClientDashboard:** Extraer en componentes independientes: RequestCreationForm, ProgressTracker, HandshakeValidation, LockerAccessPanel, RequestHistory. Cada uno con su propio archivo, tipos y tests.
2. **Descomponer DriverDashboard:** Extraer: PendingRequestsList, ActivePickupManager, HandshakeVerifier, RouteNavigator.
3. **Descomponer AdminDashboard:** Extraer: MetricsPanel, FleetStatusPanel, UserManagementPanel, AuditTrailViewer.
4. **Extraer lógica de negocio:** Mover toda lógica de negocio a custom hooks dedicados: useRequestCreation, useHandshake, useLockerAccess, useDriverPickup. Los componentes deben ser puramente presentacionales.
5. **Tipar estrictamente:** Revisar y completar las interfaces TypeScript. Eliminar todo uso de 'any'. Activar reglas ESLint estrictas para TypeScript: @typescript-eslint/no-explicit-any, @typescript-eslint/strict-boolean-expressions.

Criterios de aceptación:

- Ningún archivo de componente supera las 300 líneas de código.
- Cero usos de 'any' en todo el código TypeScript del frontend.
- Cada componente extraído tiene al menos un test unitario básico.

2.4 Hito 1.4: Estabilización del Backend

Aunque el backend está en mejor estado que el frontend, hay áreas críticas que necesitan atención antes de producción.

Tareas:

1. **Migraciones de base de datos:** Implementar un sistema formal de migraciones (utilizando db-migrate o una solución propia basada en archivos SQL versionados). La inicialización actual recrea las tablas desde cero, lo que es inaceptable en producción.
2. **Pool de conexiones:** Revisar y optimizar la configuración del pool de PostgreSQL. Configurar min/max conexiones según carga esperada. Implementar health checks de conexión.
3. **Transacciones robustas:** Auditar todos los endpoints que modifican datos y asegurar que usan transacciones PostgreSQL con ROLLBACK apropiado. Actualmente solo acceptRequest usa transacciones explícitas.
4. **Validación de entrada consistente:** Asegurar que todos los endpoints usan schemas Zod para validar el body, params y query de cada request. Centralizar los mensajes de error de validación.
5. **Variables de entorno:** Migrar todos los valores hardcoded (TTL de handshake de 5 minutos, salt rounds de bcrypt, radio de búsqueda geográfica) a variables de entorno con valores por defecto seguros. Validar todas las variables requeridas al arrancar el servidor.

Criterios de aceptación:

- Las migraciones de base de datos se ejecutan de forma incremental y son reversibles.
- Todos los endpoints críticos operan dentro de transacciones atómicas.
- El servidor no arranca si falta alguna variable de entorno crítica.

3. FASE 2: SEGURIDAD Y CUMPLIMIENTO

Duración estimada: 2-3 semanas | **Prioridad:** CRÍTICA

La seguridad es no negociable para una aplicación que maneja datos personales, ubicaciones en tiempo real y potencialmente pagos. Esta fase cubre las brechas de seguridad identificadas en la auditoría técnica.

3.1 Hito 2.1: Fortalecimiento de Autenticación

Tareas:

1. **Refresh tokens:** Implementar un sistema de refresh tokens con rotación. El access token debe tener una vida corta (15-30 minutos) y el refresh token una vida larga (7 días) almacenado como cookie HttpOnly. Esto evita la exposición prolongada de tokens en sessionStorage.
2. **Rotación de JWT secret:** Implementar soporte para múltiples JWT secrets activos simultáneamente, permitiendo rotación sin invalidar sesiones existentes.
3. **Protección contra fuerza bruta:** Implementar rate limiting específico para el endpoint de login: máximo 5 intentos por IP en 15 minutos, con bloqueo progresivo (1 min, 5 min, 15 min). Registrar intentos fallidos en audit_events.
4. **Política de contraseñas:** Mejorar la validación actual (solo 6+ caracteres) a: mínimo 8 caracteres, al menos una mayúscula, una minúscula, un número y un carácter especial. Implementar verificación contra diccionario de contraseñas comunes (top 10.000).
5. **Cierre de sesión seguro:** Implementar endpoint de logout que invalide el refresh token en servidor. Implementar cierre de sesión en todos los dispositivos.

Criterios de aceptación:

- Los access tokens expiran en 15 minutos y se renuevan transparentemente mediante refresh tokens.
- Tras 5 intentos de login fallidos, la IP queda bloqueada temporalmente con backoff progresivo.
- Las contraseñas débiles son rechazadas con mensajes específicos de qué falta.

3.2 Hito 2.2: Protección Anti-Spoofing GPS

Problema crítico: La validación de proximidad (<50m) para el handshake está documentada en la especificación pero no implementada en el código. Un conductor podría confirmar entregas sin estar físicamente presente.

Tareas:

1. **Validación de proximidad server-side:** Implementar verificación en el endpoint /confirm-driver que calcule la distancia Haversine entre la ubicación reportada del conductor y la ubicación de recogida. Rechazar si la distancia supera 100 metros (con margen para imprecisión GPS urbana).
2. **Triangulación de señales:** Almacenar las últimas 5 posiciones GPS reportadas por el conductor. Detectar saltos imposibles (velocidad >200 km/h entre dos reportes consecutivos) como indicador de spoofing.

3. **Consistencia temporal:** Verificar que las coordenadas GPS incluyen timestamp del dispositivo y que la diferencia con el tiempo del servidor no excede 30 segundos.
4. **Logging de anomalías:** Registrar todas las detecciones de posible spoofing en `audit_events` con metadatos detallados para investigación posterior.

Criterios de aceptación:

- El handshake es rechazado si el conductor reporta una ubicación a más de 100m de la recogida.
- Los saltos de GPS imposibles generan alertas en el sistema de auditoría.

3.3 Hito 2.3: Seguridad de Datos y Comunicaciones

Tareas:

5. **HTTPS obligatorio:** Configurar HSTS (HTTP Strict Transport Security) con max-age de al menos 1 año. Asegurar que toda comunicación, incluido WebSocket, use TLS 1.2+.
6. **Sanitización de datos:** Implementar sanitización de entrada contra XSS en todos los campos de texto libre (nombre, dirección de recogida). Usar bibliotecas como DOMPurify en frontend y express-validator/sanitize en backend.
7. **Protección CSRF:** Implementar tokens CSRF para todas las operaciones de escritura (POST, PUT, DELETE). Utilizar el patrón double-submit cookie.
8. **Headers de seguridad avanzados:** Revisar y ajustar la configuración de Helmet. Configurar Content-Security-Policy restrictiva que solo permita los orígenes necesarios (API, CDN de mapas Leaflet, fuentes).
9. **Encriptación de datos sensibles:** Cifrar en base de datos: códigos de locker (`access_code`), códigos de handshake (`handshake_code`). Usar AES-256-GCM con clave rotatable almacenada en variable de entorno.
10. **Autenticación de WebSockets:** Asegurar que las conexiones Socket.IO validen el token JWT en el handshake inicial y en cada reconexion. Implementar middleware de autorización que verifique que el usuario solo recibe eventos de sus propios recursos.

Criterios de aceptación:

- Cero vulnerabilidades de severidad alta en escaneo OWASP ZAP o similar.
- Los códigos sensibles están encriptados en reposo en la base de datos.
- Las conexiones WebSocket son rechazadas sin token válido.

3.4 Hito 2.4: Cumplimiento GDPR/RGPD

Al operar en España/UE, el cumplimiento del Reglamento General de Protección de Datos es obligatorio por ley.

Tareas:

6. **Política de privacidad:** Redactar e integrar política de privacidad accesible desde el registro y la app. Debe detallar: qué datos se recogen, finalidad, base legal, duración de retención, derechos del usuario.
7. **Consentimiento explícito:** Implementar checkbox de aceptación de política en el registro. Registrar el timestamp del consentimiento. Implementar consentimiento granular para geolocalización y notificaciones.
8. **Derecho al olvido:** Implementar endpoint y flujo de UI para que el usuario solicite la eliminación completa de sus datos personales. Los datos de auditoría pueden anonimizarse en lugar de eliminarse (base legal: interés legítimo / obligación legal).
9. **Exportación de datos:** Implementar endpoint que genere un archivo JSON/CSV con todos los datos personales del usuario (portabilidad de datos, Art. 20 RGPD).
10. **Retención de datos:** Definir e implementar políticas de retención: datos de solicitudes completadas se anonimizan tras 2 años, datos de geolocalización se eliminan tras 30 días, logs de auditoría se retienen 5 años.
11. **Registro de actividades de tratamiento:** Documentar formalmente todas las actividades de procesamiento de datos según Art. 30 RGPD.

Criterios de aceptación:

- Existe política de privacidad completa y accesible.
- Un usuario puede solicitar y obtener la eliminación de sus datos en menos de 72 horas.
- Los datos de geolocalización históricos se purgan automáticamente tras 30 días.

4. FASE 3: INTEGRACIONES EXTERNAS

Duración estimada: 4-5 semanas | **Prioridad:** ALTA

Esta fase transforma la aplicación de un prototipo funcional a un producto real conectado con servicios de pago, comunicación y hardware físico.

4.1 Hito 3.1: Pasarela de Pagos

La monetización es esencial para la viabilidad del negocio. Se recomienda Stripe por su excelente documentación, soporte en España y facilidad de integración.

Tareas:

6. **Modelo de datos:** Crear tablas: payments (id, request_id, amount, currency, status, stripe_payment_intent_id, created_at), pricing_rules (id, package_size, base_price, currency, active).
7. **Backend - Stripe Integration:** Implementar Stripe SDK. Crear endpoints: POST /api/payments/create-intent (crea PaymentIntent), POST /api/payments/confirm, GET /api/payments/history. Implementar webhook handler para eventos asincrónicos de Stripe (payment_intent.succeeded, payment_intent.payment_failed).
8. **Flujo de pago pre-servicio:** Integrar el pago como paso obligatorio antes de que el conductor acepte la solicitud. El PaymentIntent se crea al crear la solicitud y se captura (confirma) tras el handshake exitoso. Esto implementa el patrón auth-and-capture.
9. **Frontend - Checkout UI:** Integrar Stripe Elements (React) para un formulario de pago PCI-compliant embebido en la interfaz. Mostrar resumen de coste antes de confirmar. Soportar tarjetas de crédito/débito y Apple Pay/Google Pay.
10. **Reembolsos:** Implementar lógica de reembolso automático si la solicitud es cancelada antes de la asignación de conductor o si el conductor no se presenta en 30 minutos.
11. **Panel admin de pagos:** Añadir sección en AdminDashboard para ver transacciones, emitir reembolsos manuales y consultar métricas de ingresos.

Criterios de aceptación:

- Un cliente puede pagar con tarjeta de crédito de forma segura sin que los datos de tarjeta toquen nuestro servidor.
- El pago se captura solo después del handshake exitoso. Si se cancela antes, se reembolsa automáticamente.
- Los webhooks de Stripe se procesan de forma idempotente (re-procesar un evento no causa duplicados).

4.2 Hito 3.2: Sistema de Notificaciones SMS/Push

Actualmente los códigos OTP de handshake solo se muestran en la interfaz de la app, lo que es insuficiente para una experiencia real donde el cliente puede no tener la app abierta.

Tareas:

6. **Integración SMS (Twilio):** Implementar servicio de envío SMS vía Twilio. Enviar: código OTP de handshake al cliente, confirmación de depósito con código PIN del locker, recordatorio de recogida si no se ha recogido en 6 horas.
7. **Push Notifications (PWA):** Implementar Web Push API usando VAPID keys. Configurar el Service Worker existente (ya hay PWA con Workbox) para recibir push notifications. Enviar: nueva solicitud asignada (conductor), estado actualizado de solicitud (cliente).
8. **Sistema de templates:** Crear un sistema de plantillas de mensajes internacionalizado (ES/EN/CA como mínimo) para SMS y push. Almacenar en base de datos para edición sin redespigüe.
9. **Preferencias de notificación:** Añadir pantalla de ajustes donde el usuario configure: SMS sí/no, Push sí/no, idioma preferido.
10. **Fallback chain:** Implementar cadena de envío: intentar Push primero, si falla (usuario no ha dado permiso, dispositivo sin conexión) enviar SMS como fallback.

Criterios de aceptación:

- El cliente recibe el código OTP por SMS en menos de 10 segundos tras la generación.
- Las push notifications funcionan en segundo plano con la app cerrada.
- El usuario puede desactivar cada canal de notificación independientemente.

4.3 Hito 3.3: Integración con Lockers IoT

Actualmente el endpoint POST /lockers/:label/open es un stub que cambia el estado en base de datos pero no interactúa con hardware físico real.

Tareas:

6. **Capa de abstracción de hardware:** Crear un LockerHardwareService con interfaz genérica (open, close, getStatus, healthCheck) que permita intercambiar implementaciones según el proveedor de lockers contratado.
7. **Protocolo de comunicación:** Implementar la comunicación con el API del proveedor de lockers (REST, MQTT o protocolo propietario según fabricante). Incluir autenticación mutua TLS para seguridad de la comunicación M2M.
8. **Sincronización de estado:** Implementar polling periódico (o WebSocket/MQTT subscription) para sincronizar el estado real de cada locker con la base de datos. Detectar discrepancias (locker marcado como libre pero físicamente ocupado).
9. **Manejo de fallos de hardware:** Implementar circuit breaker para el servicio de lockers. Si el hardware no responde, marcar el locker como fuera de servicio y notificar al admin. Proporcionar código de emergencia para apertura manual.
10. **Monitorización de sensores:** Integrar datos de sensores del locker si disponibles: estado de puerta (abierta/cerrada), peso detectado, temperatura (para artículos sensibles).

Criterios de aceptación:

- El comando de apertura de locker se ejecuta en hardware real en menos de 3 segundos.
- Si un locker falla, se marca automáticamente como no disponible y se notifica al administrador.

- La capa de abstracción permite cambiar de proveedor de hardware sin modificar la lógica de negocio.

4.4 Hito 3.4: Integración con Líneas de Cruceros

El schema `cruise_manifest` ya existe pero no tiene integración real. Conectar con las líneas de cruceros permite anticipar demanda y ofrecer servicios personalizados.

Tareas:

5. **API de ingesta:** Implementar endpoint seguro (API key + whitelist de IPs) para que las navieras envíen datos de manifiestos: nombre del buque, hora de llegada, número estimado de pasajeros, terminal.
6. **Predicción de demanda:** Basado en datos históricos de pasajeros y solicitudes, implementar un algoritmo simple de predicción que estime conductores necesarios y lockers a reservar por franja horaria.
7. **Pre-reserva de lockers:** Permitir reservar bloques de lockers vinculados a un buque específico. Los lockers pre-reservados no están disponibles para el público general.
8. **Dashboard de operaciones:** Añadir vista en panel admin que muestre cruceros del día con: hora de llegada, pasajeros estimados, conductores disponibles, lockers reservados/disponibles.

Criterios de aceptación:

- Los datos de cruceros se sincronizan automáticamente al menos 24 horas antes de la llegada.
- El sistema pre-reserva lockers proporcionalmente al número de pasajeros estimado.

5. FASE 4: EXPERIENCIA DE USUARIO Y RENDIMIENTO

Duración estimada: 3-4 semanas | **Prioridad:** ALTA

Una app de logística en tiempo real debe ser rápida, intuitiva y accesible. Esta fase pule la experiencia del usuario y optimiza el rendimiento.

5.1 Hito 4.1: Accesibilidad (WCAG 2.1 AA)

La app ya tiene perfiles de accesibilidad en la base de datos (standard, PMR, age_advanced) pero no los utiliza en la interfaz.

Tareas:

1. **Auditoría de accesibilidad:** Ejecutar auditoría completa con axe-core y Lighthouse Accessibility. Documentar todas las violaciones y priorizarlas.
2. **Semántica HTML:** Revisar toda la interfaz para usar elementos semánticos correctos: nav, main, section, article, aside. Añadir roles ARIA donde sea necesario. Asegurar que todos los elementos interactivos son alcanzables por teclado (tab order).
3. **Contraste y tipografía:** Verificar ratio de contraste mínimo 4.5:1 para texto normal y 3:1 para texto grande. El estilo glass morphism actual puede tener problemas de contraste sobre fondos variables.
4. **Perfil age_advanced:** Implementar modo de interfaz simplificada para usuarios mayores: fuentes más grandes (18px base), botones más grandes (área de toque mínima 48x48px), flujo simplificado con menos pasos, instrucciones más explícitas.
5. **Perfil PMR (movilidad reducida):** Asegurar compatibilidad completa con lectores de pantalla. Implementar navegación por voz si es viable. Minimizar la necesidad de scroll y gestos complejos.
6. **Internacionalización (i18n):** Implementar react-i18next para soporte multilingüe. Idiomas mínimos para lanzamiento: Español, Inglés, Francés, Alemán, Italiano (principales nacionalidades de cruceristas en puertos españoles).

Criterios de aceptación:

- Score de Lighthouse Accessibility superior a 90.
- Toda la interfaz es navegable únicamente con teclado.
- La app funciona en al menos 5 idiomas con cambio dinámico.

5.2 Hito 4.2: Optimización de Rendimiento Frontend

Tareas:

11. **Code splitting:** Implementar lazy loading con React.lazy() y Suspense para los dashboards. Cada rol (cliente, conductor, admin) debe cargar solo su código.

12. **Optimización de mapas:** Leaflet es pesado. Implementar carga lazy del componente de mapa. Usar clustering para múltiples marcadores. Limitar la frecuencia de actualización de posición del conductor en el mapa a 1 vez por segundo (throttle).
13. **Caché inteligente con React Query:** Configurar staleTime y cacheTime apropiados para cada tipo de dato: datos estáticos (lockers, configuración) con caché larga (5 min), datos dinámicos (solicitudes activas) con caché corta (10 seg) + WebSocket para invalidación.
14. **Bundle analysis:** Analizar el bundle con vite-plugin-visualizer. Identificar y eliminar dependencias duplicadas o innecesarias. Objetivo: bundle principal <200KB gzipped.
15. **PWA mejorada:** Optimizar la estrategia de caché de Workbox: NetworkFirst para API calls, CacheFirst para assets estáticos, StaleWhileRevalidate para páginas HTML. Implementar actualización silenciosa del Service Worker con prompt al usuario.

Criterios de aceptación:

- Lighthouse Performance score superior a 85 en dispositivos móviles.
- Time to Interactive inferior a 3 segundos en conexión 3G.
- El bundle principal no supera 200KB gzipped.

5.3 Hito 4.3: Optimización de Rendimiento Backend

Tareas:

12. **Clustering de Node.js:** Implementar PM2 o el módulo cluster nativo para aprovechar múltiples cores del CPU. Esto es esencial ya que actualmente corre en un solo proceso.
13. **Caché con Redis:** Introducir Redis para: caché de consultas frecuentes (listado de lockers, métricas), almacenamiento de sesiones (para soporte multi-instancia), rate limiting distribuido, cola de mensajes para notificaciones.
14. **Optimización de queries:** Revisar y optimizar las consultas SQL más frecuentes con EXPLAIN ANALYZE. Añadir índices compuestos donde sea necesario. Implementar paginación basada en cursor para listados largos (historial, audit trail).
15. **Compresión de respuestas:** Habilitar compresión gzip/brotli en las respuestas del API. Configurar ETags para recursos que no cambian frecuentemente.
16. **Load testing:** Ejecutar y analizar resultados con el framework k6 ya presente en el proyecto. Definir benchmarks: 100 solicitudes concurrentes, tiempo de respuesta p95 < 500ms, 0 errores 5xx.

Criterios de aceptación:

- El API soporta 100 solicitudes concurrentes con p95 < 500ms.
- El uso de memoria se mantiene estable bajo carga sostenida (sin memory leaks).
- Las consultas geoespaciales resuelven en menos de 50ms.

6. FASE 5: INFRAESTRUCTURA Y DEVOPS

Duración estimada: 2-3 semanas | **Prioridad:** ALTA

Sin una infraestructura de despliegue sólida y automatizada, cada release es un riesgo. Esta fase automatiza todo el ciclo de vida del software.

6.1 Hito 5.1: Pipeline CI/CD

Actualmente la carpeta de GitHub Actions está vacía. Necesitamos automatización completa.

Tareas:

12. **Pipeline de CI (en cada Push/PR):** Lint (ESLint frontend + backend), type-check (tsc --noEmit), tests unitarios (Vitest frontend + Jest backend), build de Docker images, análisis de seguridad de dependencias (npm audit / Snyk).
13. **Pipeline de CD (en merge a main):** Build de imágenes Docker con tag semántico, push a Container Registry (ECR, GCR o GitHub Container Registry), despliegue automático a entorno de staging, ejecución de tests E2E en staging, promoción manual a producción tras aprobación.
14. **Branch protection:** Configurar reglas de protección de rama main: requerir PR, mínimo 1 review aprobado, todos los checks de CI pasados, no permitir push directo.
15. **Versionado semántico:** Implementar versionado automático basado en Conventional Commits. Usar herramientas como semantic-release o standard-version.

Criterios de aceptación:

- Ningún código llega a main sin pasar lint, tests y build exitosamente.
- El despliegue a staging es completamente automático tras merge a main.
- El despliegue a producción requiere aprobación manual explícita.

6.2 Hito 5.2: Infraestructura Cloud

Tareas:

11. **Selección de proveedor:** Evaluar AWS, GCP o DigitalOcean según coste y necesidades. Recomendación: AWS (ECS Fargate para contenedores, RDS PostgreSQL para base de datos, ElastiCache para Redis) o DigitalOcean App Platform para simplificar si el equipo es pequeño.
12. **Infraestructura como Código:** Definir toda la infraestructura con Terraform o Pulumi: VPC con subnets públicas/privadas, base de datos PostgreSQL gestionada con backups automáticos, instancias de aplicación con auto-scaling, balanceador de carga (ALB) con certificado SSL, Redis gestionado, CDN (CloudFront/Cloudflare) para assets estáticos.
13. **Entornos:** Configurar tres entornos aislados: desarrollo (para cada desarrollador, local con Docker Compose), staging (réplica de producción con datos sintéticos), producción (datos reales, configuración de alta disponibilidad).
14. **Backups y recuperación:** Configurar backups automáticos diarios de PostgreSQL con retención de 30 días. Implementar procedimiento documentado de recuperación ante desastres (DR). Objetivo: RPO < 1 hora, RTO < 4 horas.

15. **Gestión de secretos:** Migrar todos los secretos (JWT_SECRET, AUDIT_HMAC_SECRET, credenciales de Stripe, Twilio, DB) a un gestor de secretos (AWS Secrets Manager, Vault, o similar). Eliminar cualquier secreto de archivos .env en repositorios.

Criterios de aceptación:

- La infraestructura completa se puede recrear desde código en menos de 30 minutos.
- Los backups se ejecutan diariamente y se ha probado la restauración exitosamente.
- Ningún secreto existe como texto plano en el repositorio o en variables de entorno de CI.

6.3 Hito 5.3: Monitorización y Observabilidad

Tareas:

11. **Application Performance Monitoring (APM):** Integrar Sentry para tracking de errores en frontend y backend. Configurar alertas para: errores nuevos, incremento de tasa de errores >5%, errores en flujos críticos (pago, handshake).
12. **Métricas de infraestructura:** Configurar Prometheus + Grafana (o equivalente cloud-native) para monitorizar: CPU, memoria, disco, latencia de red, conexiones de DB, cola de mensajes.
13. **Métricas de negocio:** Crear dashboard de KPIs en tiempo real: solicitudes por hora, tiempo medio de asignación, tasa de handshakes exitosos vs fallidos, ocupación de lockers, ingresos por hora/día.
14. **Alerting:** Configurar alertas críticas con escalación: PagerDuty/OpsGenie para alertas P1 (servicio caído, 0 conductores activos), Slack para alertas P2 (degradación de rendimiento, locker fuera de servicio), email diario con resumen de salud del sistema.
15. **Logging centralizado:** Configurar ELK stack (Elasticsearch, Logstash, Kibana) o alternativa gestionada (DataDog, AWS CloudWatch) para agregar logs de todos los servicios. El backend ya usa Pino con formato JSON estructurado, lo que facilita la ingesta.
16. **Health checks:** Implementar endpoints /health y /ready en el backend que verifiquen: conexión a DB, conexión a Redis, estado de servicios externos (Stripe, Twilio). Usar para readiness probes en orquestador.

Criterios de aceptación:

- Cualquier error en producción genera una alerta en menos de 1 minuto.
- Existe un dashboard con KPIs de negocio actualizados en tiempo real.
- Los logs son buscables y correlacionables entre frontend y backend.

7. FASE 6: TESTING INTEGRAL Y QA

Duración estimada: 3-4 semanas | **Prioridad:** CRÍTICA

Esta fase es el último filtro antes de producción. Ningún software debe lanzarse sin una cobertura de testing exhaustiva.

7.1 Hito 6.1: Tests Unitarios del Frontend

Estado actual: menos del 10% de cobertura. Objetivo: superar el 80%.

Tareas:

9. **Tests de componentes UI:** Escribir tests con Testing Library para cada componente extraído en la Fase 1. Cubrir: renderizado correcto, manejo de props, estados de carga, estados de error, interacciones del usuario (clicks, inputs, submits).
10. **Tests de hooks:** Testear cada custom hook con @testing-library/react-hooks: useSocket (conexión, desconexión, eventos), useClientGeoLocation (permisos, coordenadas, errores), useRequestCreation (validación, envío, estados).
11. **Tests de servicios:** Testear la capa de API (services/api.ts) con mocks de Axios. Verificar: construcción correcta de URLs, headers de autenticación, manejo de errores HTTP, transformación de respuestas.
12. **Snapshot tests:** Implementar snapshot tests para componentes críticos de UI (formularios, tarjetas de solicitud, paneles de locker) para detectar cambios visuales no intencionados.

Criterios de aceptación:

- Cobertura de tests del frontend superior al 80% en líneas y funciones.
- Todos los flujos críticos del usuario tienen tests que cubren happy path y error path.

7.2 Hito 6.2: Tests End-to-End (E2E)

Los tests E2E simulan el comportamiento real del usuario en un navegador. Son la última línea de defensa antes de producción.

Tareas:

7. **Setup de Playwright:** Configurar Playwright (preferido sobre Cypress por soporte multi-navegador nativo). Configurar para ejecutar en: Chromium, Firefox y WebKit (Safari). Integrar con CI pipeline.
8. **Flujos críticos del cliente:** Registro de nuevo cliente, login, crear solicitud de recogida con geolocalización, ver progreso en tiempo real, recibir código de locker, consultar historial.
9. **Flujos críticos del conductor:** Login como conductor, ver solicitudes pendientes, aceptar solicitud, navegar a ubicación, completar handshake, depositar en locker.
10. **Flujos críticos del admin:** Login como admin, verificar métricas, gestionar usuarios, consultar audit trail, monitorizar flota.

11. **Tests de integración completa:** Simular el flujo completo de una solicitud desde perspectiva de cliente Y conductor simultáneamente (usando dos contextos de navegador en paralelo). Verificar que las actualizaciones en tiempo real funcionan entre ambos.
12. **Visual regression:** Configurar Playwright para capturas de pantalla y comparación visual automática. Detectar cambios no intencionados en la UI.

Criterios de aceptación:

- Al menos 30 tests E2E cubriendo todos los flujos críticos.
- Los tests pasan en Chromium, Firefox y WebKit.
- Los tests se ejecutan en el pipeline de CI en cada PR.

7.3 Hito 6.3: Tests de Seguridad

Tareas:

16. **Penetration testing automatizado:** Ejecutar OWASP ZAP o Burp Suite Community contra la app en staging. Documentar y corregir todas las vulnerabilidades encontradas.
17. **Dependency audit:** Configurar npm audit y Snyk en CI. Política: cero vulnerabilidades críticas o altas permitidas, las medias deben resolverse en 30 días.
18. **Tests de autorización:** Escribir tests específicos que verifiquen que: un cliente no puede acceder a endpoints de conductor, un conductor no puede acceder a endpoints de admin, un usuario no puede ver/modificar recursos de otro usuario (IDOR).
19. **Tests de rate limiting:** Verificar que los límites de tasa se aplican correctamente bajo carga: login (5/15min), API general (100/15min), handshake (3 intentos).

Criterios de aceptación:

- Cero vulnerabilidades críticas o altas en el informe OWASP ZAP.
- Todas las pruebas de autorización pasan: ningún rol puede escalar privilegios.

7.4 Hito 6.4: Tests de Carga y Estrés

El framework k6 ya está presente en el proyecto. Es momento de usarlo seriamente.

Tareas:

17. **Escenarios de carga:** Definir y ejecutar: carga normal (50 usuarios simultáneos), carga pico (200 usuarios, simulando llegada de crucero), estrés (incremento hasta encontrar el punto de ruptura), resistencia (carga sostenida durante 2 horas para detectar memory leaks).
18. **Benchmarks por endpoint:** Medir latencia p50, p90, p95 y p99 para cada endpoint crítico. Establecer SLOs: p95 < 500ms para operaciones normales, p95 < 200ms para consultas de lectura.
19. **WebSocket load testing:** Testear la capacidad de conexiones WebSocket simultáneas. Verificar que las actualizaciones en tiempo real se entregan con latencia <1 segundo bajo carga.

20. **Informe de capacidad:** Documentar la capacidad máxima del sistema y las recomendaciones de escalado. Definir umbrales de auto-scaling basados en los resultados.

Criterios de aceptación:

- El sistema soporta 200 usuarios simultáneos sin degradación significativa ($p95 < 1s$).
- No hay memory leaks detectados tras 2 horas de carga sostenida.
- Se ha documentado el punto de ruptura y el plan de escalado.

8. FASE 7: LANZAMIENTO Y POST-LANZAMIENTO

Duración estimada: 2-3 semanas | **Prioridad:** ALTA

La fase final prepara el lanzamiento controlado y establece las bases para el crecimiento sostenible post-lanzamiento.

8.1 Hito 7.1: Preparación Pre-Lanzamiento

Tareas:

16. **Checklist de producción:** Verificar: todas las variables de entorno de producción configuradas, JWT_SECRET generado con mínimo 64 caracteres aleatorios, CORS configurado solo para el dominio de producción, modo debug/demo deshabilitado, cuentas de demo eliminadas de los datos semilla, rate limiting ajustado para carga real.
17. **Documentación operativa:** Crear runbook con procedimientos para: despliegue y rollback, escalado manual de emergencia, recuperación ante caída de base de datos, rotación de secretos, respuesta ante incidentes de seguridad.
18. **Términos de servicio:** Redactar y publicar términos de servicio, política de privacidad (del Hito 2.4), y política de cookies. Integrar banner de consentimiento de cookies.
19. **Dominio y SSL:** Configurar dominio de producción con certificado SSL (Let's Encrypt o certificado comercial). Configurar DNS con registros A, CNAME, y registros MX si se envían emails transaccionales.
20. **SEO y metadatos:** Configurar metatags, Open Graph, favicon, manifest.json para la PWA. Asegurar que la app es instalable como PWA en iOS y Android.

Criterios de aceptación:

- La checklist de producción está 100% completada y verificada tanto por Luis como por Pablo de forma independiente.
- El runbook operativo está documentado y ha sido validado con un simulacro de incidente.

8.2 Hito 7.2: Lanzamiento Beta Controlado

Tareas:

16. **Beta cerrada:** Lanzar con un grupo controlado de 20-50 usuarios reales (conductores reclutados + clientes invitados). Durará 2 semanas. Recopilar feedback activamente.
17. **Feature flags:** Implementar sistema de feature flags (LaunchDarkly, Unleash o solución propia simple) para poder activar/desactivar funcionalidades sin redespliegue. Especialmente útil para: sistema de pagos, notificaciones SMS, integración con lockers reales.
18. **Canal de feedback:** Implementar botón de feedback in-app que permita a los beta testers reportar bugs y sugerencias directamente. Integrar con herramienta de tracking (Linear, Jira, GitHub Issues).
19. **Monitorización intensiva:** Durante la beta, mantener monitorización 24/7 con rotación de guardia. Revisar métricas y logs diariamente. Responder a incidentes en menos de 30 minutos.

Criterios de aceptación:

- Al menos 50 transacciones completadas exitosamente durante la beta.
- Tasa de errores inferior al 2% sobre el total de operaciones.
- NPS (Net Promoter Score) de los beta testers superior a 30.

8.3 Hito 7.3: Lanzamiento Público (GA)**Tareas:**

17. **Lanzamiento progresivo:** Abrir el servicio gradualmente: primero un solo puerto (Las Palmas como está configurado), aumentar la capacidad de conductores progresivamente, monitorizar carga y escalar según demanda.
18. **Onboarding de usuarios:** Implementar tutorial interactivo para nuevos usuarios (cliente y conductor) que guíe por el primer uso de la app. Implementar tooltips contextuales en puntos clave del flujo.
19. **Soporte al cliente:** Establecer canal de soporte: chat in-app (Intercom, Zendesk o similar), email de soporte, FAQ dentro de la aplicación.
20. **Analytics:** Integrar analytics (Mixpanel, Amplitude o Google Analytics 4) para tracking de: funnel de conversión (registro → primera solicitud → solicitud completada), retención de usuarios, tiempo medio por paso del flujo, mapas de calor de uso.

Criterios de aceptación:

- La app está disponible públicamente y procesa solicitudes reales.
- Existe canal de soporte con SLA de respuesta definido (<4 horas en horario laboral).
- Los analytics están activos y el equipo revisa métricas semanalmente.

8.4 Hito 7.4: Plan Post-Lanzamiento

El lanzamiento no es el final, sino el inicio de un ciclo continuo de mejora.

Tareas:

13. **Sistema de merchants B2B:** Completar la funcionalidad de merchants que ya tiene schema y endpoints parciales. Permitir que tiendas se registren y ofrezcan el servicio de entrega en locker directamente a sus clientes.
14. **Expansión multi-puerto:** Adaptar la configuración para soportar múltiples puertos simultáneamente. Cada puerto con sus propios lockers, conductores y configuración de geofencing. Barcelona, Málaga, Palma de Mallorca como siguientes destinos.
15. **App nativa (React Native):** Evaluar la migración de la PWA a una app nativa con React Native o Expo para mejorar: acceso a GPS en background, push notifications nativas, presencia en App Store y Google Play.
16. **Programa de fidelización:** Implementar sistema de puntos/descuentos para usuarios recurrentes y conductores con alta calificación.

17. **Integración con plataformas de turismo:** API para integración con plataformas de reserva de excursiones, hoteles y servicios portuarios.

Criterios de aceptación:

- Plan de producto a 12 meses documentado y priorizado.
- Roadmap público disponible para stakeholders y usuarios clave.

9. CRONOGRAMA CONSOLIDADO

La siguiente tabla muestra la estimación temporal total del proyecto. Las fases pueden solaparse parcialmente cuando no existen dependencias directas entre sus hitos.

Fase	Semanas	Inicio Est.	Fin Est.	Dependencias
F1: Estabilización	3-4	Semana 1	Semana 4	Ninguna
F2: Seguridad	2-3	Semana 3	Semana 6	F1 parcial
F3: Integraciones	4-5	Semana 5	Semana 10	F1, F2
F4: UX y Rendimiento	3-4	Semana 7	Semana 11	F1
F5: DevOps	2-3	Semana 5	Semana 8	F1 parcial
F6: Testing y QA	3-4	Semana 9	Semana 13	F1-F5
F7: Lanzamiento	2-3	Semana 13	Semana 16	F1-F6

Duración total estimada: 16-20 semanas (4-5 meses) con Luis y Pablo trabajando como equipo a tiempo completo. Con buena coordinación y solapamiento óptimo entre fases independientes, se puede comprimir a 16-18 semanas.

9.1 Equipo: Luis y Pablo

El proyecto será ejecutado íntegramente por Luis y Pablo, que deberán compenetrarse como equipo para cubrir todas las áreas del desarrollo. A continuación se detalla la distribución de responsabilidades recomendada, aunque ambos deben tener conocimiento completo de todo el sistema para poder apoyarse mutuamente.

Responsabilidad	Asignación Principal	Apoyo / Backup	Fases
Arquitectura y decisiones técnicas	Luis y Pablo (conjunto)	—	Todas
Frontend (React, UI, accesibilidad)	Pablo	Luis	F1, F4, F6, F7
Backend (API, DB, servicios)	Luis	Pablo	F1, F2, F3, F5
Integraciones externas (Stripe, Twilio, IoT)	Luis	Pablo	F3
Seguridad y cumplimiento RGPD	Luis y Pablo (conjunto)	—	F2
DevOps, CI/CD e infraestructura	Luis	Pablo	F5
Testing (unitarios, E2E, carga)	Pablo	Luis	F6
UX/UI y accesibilidad	Pablo	Luis	F4

Responsabilidad	Asignación Principal	Apoyo / Backup	Fases
Documentación y runbooks	Luis y Pablo (conjunto)	—	F7
QA final y lanzamiento	Luis y Pablo (conjunto)	—	F6, F7

9.2 Dinámica de Trabajo Luis-Pablo

Para que un equipo de dos personas pueda abordar un proyecto de esta envergadura con éxito, es fundamental establecer una dinámica de trabajo eficiente:

20. **Pair programming en decisiones críticas:** Las tareas de arquitectura, seguridad y los flujos transaccionales críticos deben trabajarse juntos. El pair programming en estos puntos evita errores costosos y asegura que ambos entienden el sistema completo.
21. **Trabajo en paralelo en fases independientes:** Mientras Luis trabaja en integraciones backend (Stripe, Twilio), Pablo puede avanzar en refactorización de componentes frontend. La clave es definir interfaces (contratos de API) antes de separarse.
22. **Code reviews cruzados obligatorios:** Todo código que uno escriba debe ser revisado por el otro antes de mergearse. Sin excepciones. Esto es el principal mecanismo de calidad en un equipo pequeño.
23. **Daily standup de 15 minutos:** Cada mañana, sincronización rápida de: qué hice ayer, qué haré hoy, qué me bloquea. En un equipo de dos esto es aún más crítico para evitar desalineación.
24. **Rotación de conocimiento:** Cada 2 semanas, Luis y Pablo deben intercambiar brevemente sus áreas principales (Luis hace tareas de frontend, Pablo de backend) para asegurar que ninguno sea un punto único de fallo. Si uno enferma o no está disponible, el otro debe poder continuar.
25. **Planificación semanal conjunta:** Cada lunes, revisión del progreso de la fase actual, ajuste de prioridades y distribución de tareas de la semana. Usar un tablero Kanban compartido (GitHub Projects, Linear o Trello).
26. **Gestión de conocimiento externo:** Para las áreas donde ninguno tenga experiencia profunda (por ejemplo DevOps avanzado, diseño UX, legal RGPD), planificar consultas puntuales con profesionales externos o utilizar recursos de formación específicos antes de abordar esas tareas.

10. GESTIÓN DE RIESGOS

A continuación se identifican los principales riesgos del proyecto con su impacto, probabilidad y estrategia de mitigación.

Riesgo	Impacto	Prob.	Mitigación
Retraso en integración IoT de lockers	Alto	Media	Mantener capa de abstracción. Lanzar con lockers manuales (código + candado) como fallback.
Cambios regulatorios RGPD	Alto	Baja	Consultar con abogado especialista. Implementar arquitectura flexible para retención/eliminación.
Baja adopción de conductores	Alto	Media	Programa de incentivos económicos durante primeros 3 meses. Simplificar onboarding al máximo.
Escalabilidad insuficiente en pico	Medio	Media	Load testing exhaustivo en F6. Auto-scaling configurado. Plan de escalado manual de emergencia.
Fallos de GPS en zonas portuarias	Medio	Alta	Validación de proximidad con margen amplio (100m). Opción manual de confirmación como fallback.
Dependencia de proveedor único (Stripe/Twilio)	Medio	Baja	Capa de abstracción para pagos y SMS. Evaluar alternativas: Adyen, Vonage.
Indisponibilidad de Luis o Pablo	Alto	Media	Rotación de conocimiento cada 2 semanas. Ambos deben conocer todo el sistema. Documentación y tests como red de seguridad.

11. MÉTRICAS DE ÉXITO (KPIs)

Estas métricas deben monitorizarse continuamente para evaluar el progreso y la salud del producto.

11.1 KPIs Técnicos

Métrica	Objetivo	Herramienta
Cobertura de tests (backend)	> 85%	Jest + Istanbul
Cobertura de tests (frontend)	> 80%	Vitest + c8
Tests E2E pasando	100% en CI	Playwright
Vulnerabilidades críticas	0	OWASP ZAP + Snyk
Uptime	> 99.5%	UptimeRobot / Pingdom
Latencia API p95	< 500ms	Prometheus + Grafana
Tiempo de despliegue	< 10 min	GitHub Actions
Tasa de errores 5xx	< 0.1%	Sentry

11.2 KPIs de Negocio

Métrica	Objetivo Beta	Objetivo GA
Solicitudes completadas/día	5-10	50+
Tiempo de asignación	< 2 min	< 1 min
Tiempo total del servicio	< 45 min	< 30 min
Tasa de handshake exitoso	> 90%	> 97%
NPS de usuarios	> 30	> 50
Retención a 30 días	> 40%	> 60%
Conductores activos diarios	5-10	30+
Ocupación media de lockers	20-40%	60-80%